CLAW
SCREENWRITER

clawscreenwriter.com

Complete Setup Guide

OpenClaw · Cursor · Supabase · Vercel

What You Are Building	What You Need
A live screenwriting canvas	Mac computer
AI that writes autonomously into your script	Internet connection
Real-time sync — changes appear in 1 second	Anthropic account (console.anthropic.com)
Deployed live at clawscreenwriter.com	GitHub account (free)
OpenClaw writes entire acts on command	About 60–90 minutes to complete setup

WARNING
Follow every step in exact order. Do not skip ahead. Each part builds on the one before it.

At a Glance — The 6 Parts

Part	Title	What Happens
1	Install Everything	Homebrew, Node, Git, Cursor, Claude Code
2	Supabase Setup	Create database, run schema, get API keys
3	Build the Project	Create Vite app, install dependencies, env file
4	Run OpenClaw	Feed 5 build prompts, QA test all features
5	Deploy to Vercel	Push to GitHub, go live on Vercel
6	Start Writing	How to launch an OpenClaw writing session

PART 1 Install Everything

Homebrew · Node.js · Git · Cursor · Claude Code · GitHub

Open the Terminal application on your Mac. You can find it by pressing Cmd + Space and typing Terminal. Every command in this section is typed into Terminal and confirmed with the Enter key.

STEP 1 Install Homebrew

Homebrew is a package manager for Mac. It installs developer tools cleanly.
Paste this into Terminal and press Enter:

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

When it finishes, paste this second command and press Enter:

```
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv)"
```

STEP 2 Install Node.js

Node is required to run the project and install packages.

```
brew install node
```

Verify both installed correctly:

```
node --version
npm --version
```

TIP

Both commands should print a version number like v20.11.0 and 10.2.4. If you see command not found, close Terminal and reopen it, then try again.

STEP 3 Install Git

Git manages your code and connects to GitHub for deployment.

```
brew install git
```

Configure it with your name and email:

```
git config --global user.name "Spencer Longmore"
git config --global user.email "your@email.com"
```

STEP 4 Download and Install Cursor

Cursor is the code editor where OpenClaw will build the platform.

1. Go to cursor.com
2. Click Download
3. Open the .dmg file
4. Drag Cursor into your Applications folder
5. Open Cursor from Applications
6. Sign in or create a free account
7. When asked about AI model — select Claude Sonnet
8. Skip all other onboarding steps

STEP 5 Install Cursor CLI

This lets you open Cursor directly from Terminal.
Inside Cursor, press Cmd + Shift + P
Type: Shell Command: Install cursor command in PATH
Click it when it appears in the dropdown.

STEP 6 Get Your Anthropic API Key

This key is what powers OpenClaw.

1. Go to console.anthropic.com
2. Sign in with your Anthropic account
3. Click API Keys in the left sidebar
4. Click Create Key
5. Name it: openclaw
6. Copy the key — it starts with sk-ant-
7. Save it somewhere safe. You will not see it again.

STEP 7 Set Your API Key in Terminal

This saves the key so OpenClaw can use it automatically.

```
echo 'export ANTHROPIC_API_KEY="sk-ant-YOUR-KEY-HERE"' >> ~/.zprofile
source ~/.zprofile
```

Verify it saved correctly:

```
echo $ANTHROPIC_API_KEY
```

WARNING

This should print your full API key. If it prints nothing, repeat Step 7 carefully and make sure you replaced sk-ant-YOUR-KEY-HERE with your actual key.

STEP 8 Install Claude Code — OpenClaw's Engine

```
npm install -g @anthropic-ai/claude-code
```

Verify it installed:

```
claude --version
```

STEP 9 Create a GitHub Account

You need GitHub to deploy the site to Vercel.
Go to github.com and create a free account if you do not have one.
Remember your username — you will need it shortly.

STEP 10 Install GitHub CLI and Authenticate

```
brew install gh
```

Authenticate with your GitHub account:

```
gh auth login
```

Follow the prompts in this exact order:

- Select GitHub.com
- Select HTTPS
- Select Login with a web browser
- Copy the 8-character code it gives you
- Press Enter — your browser opens automatically
- Paste the code and click Authorize

TIP
Part 1 is complete. Every tool is installed. Close Terminal — you will use the terminal inside Cursor for everything from here.

PART 2 Supabase Setup

Database · Schema · Real-Time · API Keys

Supabase is the database that stores every screenplay document and powers the real-time sync. When OpenClaw writes a scene, Supabase broadcasts the change and your browser updates within one second.

STEP 11 Create Your Supabase Project

1. Go to supabase.com
2. Create a free account or sign in
3. Click New Project
4. Fill in the details:
   - Name: clawscreenwriter
   - Database password: create a strong one and save it
   - Region: pick the one closest to you
5. Click Create new project
6. Wait approximately 2 minutes for it to spin up

STEP 12 Run the Database Schema

1. In Supabase, click SQL Editor in the left sidebar
2. Click New Query
3. Copy the entire block below and paste it into the editor
4. Click Run
5. You should see: Success. No rows returned.

PASTE INTO SUPABASE SQL EDITOR:

```
CREATE TABLE documents (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project TEXT NOT NULL,
  section TEXT NOT NULL,
  type TEXT NOT NULL,
  name TEXT,
  content TEXT DEFAULT '',
  parent_id UUID REFERENCES documents(id) ON DELETE SET NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

```
CREATE INDEX idx_documents_project ON documents(project);
CREATE INDEX idx_documents_type ON documents(project, section, type);

CREATE OR REPLACE FUNCTION update_updated_at()
RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = NOW();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER set_updated_at
BEFORE UPDATE ON documents
FOR EACH ROW EXECUTE FUNCTION update_updated_at();

ALTER PUBLICATION supabase_realtime ADD TABLE documents;

INSERT INTO documents (project, section, type, name, content) VALUES
('The Last Broadcast', 'DEVELOPMENT', 'title', 'Title', ''),
('The Last Broadcast', 'DEVELOPMENT', 'logline', 'Logline', ''),
('The Last Broadcast', 'DEVELOPMENT', 'tagline', 'Tagline', ''),
('The Last Broadcast', 'DEVELOPMENT', 'brief_summary', 'Brief Summary',
''),
('The Last Broadcast', 'DEVELOPMENT', 'synopsis', 'Synopsis', ''),
('The Last Broadcast', 'DEVELOPMENT', 'treatment', 'Treatment', ''),
('The Last Broadcast', 'DEVELOPMENT', 'outline', 'Outline', ''),
('The Last Broadcast', 'DEVELOPMENT', 'beat_sheet', 'Beat Sheet', ''),
('The Last Broadcast', 'SCREENPLAY', 'script', 'Draft 1', '');
```

STEP 13 ## Copy Your API Keys

1. In Supabase, click Settings in the left sidebar
2. Click API
3. Copy and save BOTH of these — you need them in Part 3:

Project URL
Looks like: https://abcdefghijkl.supabase.co

anon public key
Long string starting with eyJ — about 200 characters

WARNING

Save both keys in a Notes file right now. You will paste them in Part 3 and again when deploying to Vercel. Do not close this Supabase tab yet.

PART 3 Build the Project

Vite · React · TypeScript · Tailwind · Environment File

STEP 14 Create the Project Folder

Open Terminal (not Cursor yet). Run these commands:

```
cd ~/Desktop
mkdir clawscreenwriter
cd clawscreenwriter
```

STEP 15 Open in Cursor

```
cursor .
```

Cursor opens with your empty project folder. From this point forward, use the terminal inside Cursor — not the Mac Terminal application.

STEP 16 Open Terminal Inside Cursor

Press Ctrl + ` (that is the backtick key, top-left of keyboard, under Escape)
A terminal panel opens at the bottom of Cursor.
Use this terminal for all remaining commands.

STEP 17 Create the Vite Project and Install Dependencies

Paste these commands one at a time into the Cursor terminal:

Command 1 — Create the project:

```
npm create vite@latest . -- --template react-ts
```

NOTE
When it asks about existing files — select Ignore files and continue.

Command 2 — Install base dependencies:

```
npm install
```

Command 3 — Install platform dependencies:

```
npm install @supabase/supabase-js @uiw/react-codemirror
@codemirror/lang-markdown lucide-react date-fns uuid @types/uuid
```

Command 4 — Install and configure Tailwind:

```
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

STEP 18 Create the Environment File

This file tells the app your Supabase credentials.
Paste this into the Cursor terminal:

```
cat > .env << 'EOF'
VITE_SUPABASE_URL=YOUR_PROJECT_URL_HERE
VITE_SUPABASE_ANON_KEY=YOUR_ANON_KEY_HERE
EOF
```

Now open the .env file in Cursor and replace both placeholders with your actual values from Step 13.

Then add .env to your gitignore so your keys are never uploaded to GitHub:

```
echo ".env" >> .gitignore
```

STEP 19 Verify the Project Runs

```
npm run dev
```

Open your browser and go to http://localhost:5173. You should see a basic Vite + React page. This confirms the foundation is working.

Press Ctrl + C in the terminal to stop it before moving to Part 4.

PART 4 Run OpenClaw to Build Everything

5 Prompts · Wait for Completion Between Each One

This is where OpenClaw takes over. You are going to start Claude Code and feed it 5 sequential prompts. OpenClaw will build the entire platform — every component, every hook, every database function — by reading these instructions and writing the code directly into your project files.

WARNING

Wait for OpenClaw to finish and confirm completion before pasting the next prompt. Do not rush this. Each prompt builds on the one before it.

STEP 20 Start Claude Code

In the Cursor terminal, type:

```
claude
```

You are now inside Claude Code. This is OpenClaw's engine. You are talking directly to Claude with full access to your project files. Everything from here is typed into this interface.

Prompt 1 — Types, Database, and Fountain Parser

Copy the entire block below and paste it into the Claude Code prompt. Press Enter and wait for it to finish.

OPENCLAW PROMPT 1 — Copy and paste this entire block:

```
You are building Claw Screenwriter, a screenwriting platform.
The project is already scaffolded with Vite + React +
TypeScript + Tailwind. Supabase is installed.
The .env file has VITE_SUPABASE_URL and
VITE_SUPABASE_ANON_KEY already set.

Create these files completely:

src/types/index.ts:
export type Section = 'DEVELOPMENT' | 'SCREENPLAY'
```

```
export type DocType =
  | 'title' | 'logline' | 'tagline' | 'brief_summary'
  | 'synopsis' | 'treatment' | 'outline'
  | 'beat_sheet' | 'script'
export interface Document {
  id: string
  project: string
  section: Section
  type: DocType
  name: string | null
  content: string
  parent_id: string | null
  created_at: string
  updated_at: string
}

src/lib/supabase.ts:
import { createClient } from '@supabase/supabase-js'
export const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL,
  import.meta.env.VITE_SUPABASE_ANON_KEY
)

src/lib/db.ts — implement and export:
- getProjects(): Promise<string[]>
- getDocumentsByProject(project): Promise<Document[]>
- getDocument(id): Promise<Document | null>
- updateDocument(id, content): Promise<void>
- createDocument(project, section, type, name,
    content, parentId?): Promise<Document>
- createVersion(parentId, content, name?): Promise<Document>
  Auto-increment name: Draft 1 -> Draft 2, V1 -> V2
- getVersionHistory(documentId): Promise<Document[]>
- deleteDocument(id): Promise<void>
- renameDocument(id, name): Promise<void>
- createProjectWithDefaults(name): Promise<Document[]>
  Creates all 9 default documents, returns array

src/lib/fountain.ts — implement parseFountain(text):
Returns { type, text }[] where type is one of:
slug | action | character | parenthetical |
dialogue | transition | note
Export as both default and named export.

Confirm all files have no TypeScript errors.
```

Prompt 2 — Hooks

Wait for Prompt 1 to complete, then paste this:

OPENCLAW PROMPT 2:

```
Continue building Claw Screenwriter.
Create these three hooks completely:

src/hooks/useDocuments.ts
Input: project string
Returns: { documents, loading, refetch }
- Fetch on mount via getDocumentsByProject(project)
- Supabase real-time channel filtered by project
- On any postgres_changes event: call refetch()
- Clean up channel on unmount
- Re-run when project prop changes

src/hooks/useDocument.ts
Input: id string | null
Returns: { document, loading, content, setContent,
  externalUpdate, acceptExternalUpdate,
  dismissExternalUpdate }
- Real-time subscription on specific row by id
- When UPDATE arrives:
  if local === lastServerContent: apply silently
  if they differ: set externalUpdate, do not apply
- acceptExternalUpdate: apply and clear
- dismissExternalUpdate: clear only
- Auto-dismiss after 10 seconds

src/hooks/useRealtime.ts
Input: project string, onUpdate: () => void
- Subscribe to all changes filtered by project
- Call onUpdate on INSERT, UPDATE, DELETE
- Clean up on unmount
```

Prompt 3 — All Components

Wait for Prompt 2 to complete, then paste this:

OPENCLAW PROMPT 3:

```
Continue building Claw Screenwriter.
Create all components. Use CSS variables throughout.

CSS variables:
--sidebar-bg: #111113  --sidebar-hover: #1e1e22
--editor-bg: #0e0e10   --panel-bg: #131316
--border: rgba(255,255,255,0.07)
--text-primary: #e8e6e0  --text-secondary: #7a7870
--text-muted: #4a4845   --accent: #4f7fff
--accent-dim: rgba(79,127,255,0.15)
```

```
--gold: #c9a84c  --green: #4a9e6e
--font-ui: 'DM Sans'  --font-script: 'Crimson Pro'
--font-mono: 'JetBrains Mono'

src/components/PreviewPane.tsx
Parse with parseFountain. Background #f4f2ec.
Courier New 11.5px. Color #1a1a1a. Padding 28px 32px.
Screenplay formatting: slug bold uppercase,
character centered, dialogue indented.

src/components/FountainSplitEditor.tsx
Left pane: textarea, Fountain syntax colors.
Right pane: PreviewPane component.
split: 50/50 | write: left only | preview: right only

src/components/SceneNav.tsx
Width 180px. Two tabs: Scenes and Cast.
Scenes: numbered slug list, clickable.
Cast: character names with scene count badge.

src/components/DocumentHeader.tsx
Height 44px. Breadcrumb left. Actions right.
Save indicator, version dropdown, Split/Write/Preview
toggle (script only), New Version, PDF buttons.

src/components/StatusBar.tsx
Height 28px. Type + version, word count, scene count,
Auto-save on, OpenClaw Watching, DB Connected.

src/components/TreeItem.tsx
Depth-based padding, active blue border,
right-click context menu, inline rename,
version and archived badges.

src/components/Sidebar.tsx
Width 248px. Project dropdown. DEVELOPMENT and
SCREENPLAY sections with dots. Footer buttons:
+ New File, + Project, Export.

src/components/Editor.tsx
useDocument hook. Debounced save 1500ms.
External update banner: Accept / Keep Mine,
auto-dismiss 10 seconds.
title/logline/tagline: styled inputs.
synopsis/treatment/etc: Crimson Pro textarea.
script: FountainSplitEditor + SceneNav.
StatusBar at bottom.
```

Prompt 4 — App Shell and Global Styles

Wait for Prompt 3 to complete, then paste this:

OPENCLAW PROMPT 4:

```
Continue building Claw Screenwriter.
Complete the app shell and all global styles.

src/index.css — replace entirely with:
@import Google Fonts: Crimson Pro, JetBrains Mono,
DM Sans (all weights needed).
Set all CSS variables listed in Prompt 3.
html/body/root: height 100%, overflow hidden,
background var(--editor-bg), color var(--text-primary).
Custom scrollbars: 4px, var(--border-md) thumb.
Print media: hide sidebar, scene-nav, document-header,
status-bar. Show preview-pane full width white.

tailwind.config.js — extend with:
fontFamily: ui, script, mono families.

src/App.tsx:
State: activeProject, activeDocumentId, projects.
On mount: getProjects(), set first project,
auto-select title document by id.
Layout: display flex, height 100vh, overflow hidden.
Render Sidebar and Editor side by side.

Create openclaw-api.md in project root.
Document the full Supabase REST API for OpenClaw:
headers, GET, PATCH, POST, DELETE operations,
using the actual values from the .env file.
Include the full write workflow.

Run npm run dev.
Confirm app loads at localhost:5173.
Confirm The Last Broadcast appears in sidebar.
Confirm clicking documents opens them in editor.
```

Prompt 5 — QA and Fix

Wait for Prompt 4 to complete. This is the final build prompt. OpenClaw will test the entire platform and fix every issue it finds.

OPENCLAW PROMPT 5 — Quality Assurance:

```
Claw Screenwriter is built. Run QA.
Test each item and fix all issues before confirming done.
```

```
Test in this exact order:

1. npm run dev — loads at localhost:5173, no errors
2. Sidebar shows The Last Broadcast with all 9 documents
   in correct DEVELOPMENT and SCREENPLAY sections
3. Click Title — single-line input opens, Crimson Pro font
4. Type something — Saving... then Saved indicator works
5. Refresh page — content persisted from Supabase
6. Click Synopsis — textarea opens, Crimson Pro font
7. Click Script — Fountain split editor opens
8. In script editor type: INT. COFFEE SHOP — DAY
   Confirm bold/white in editor
   Confirm appears in preview pane
   Confirm appears in Scenes tab of scene nav
9. Right-click Synopsis — context menu appears
   New Version — Synopsis V2 appears in sidebar
   Version dropdown shows both versions
   Switching versions loads correct content
10. Click + Project — enter new name
    All 9 documents created, project in dropdown
11. Click PDF button on script
    Print dialog opens, only preview pane visible

Fix every issue found.
Do not confirm done until all 11 tests pass cleanly.
```

TIP

When OpenClaw confirms all tests pass, Part 4 is complete. The platform is fully built and running locally.

PART 5 Deploy to Vercel

GitHub · Vercel · Live URL

The platform works locally. Now you deploy it so it is live on the internet at your domain. This takes about 5 minutes.

STEP 22 Push to GitHub

In the Cursor terminal (type exit first if still in Claude Code):

```
git init
git add .
git commit -m "Claw Screenwriter initial build"
gh repo create clawscreenwriter --public --source=. --push
```

NOTE

This creates a public GitHub repository called clawscreenwriter and pushes all your code to it.

STEP 23 Deploy to Vercel

1. Go to vercel.com
2. Sign in with your GitHub account
3. Click Add New Project
4. Find clawscreenwriter in your repository list
5. Click Import
6. BEFORE clicking Deploy — click Environment Variables
7. Add these two variables:

   Key: VITE_SUPABASE_URL
   Value: your Supabase Project URL

   Key: VITE_SUPABASE_ANON_KEY
   Value: your Supabase anon key

8. Click Deploy
9. Wait approximately 90 seconds
10. Vercel gives you a live URL

STEP 24 Connect Your Domain

1. In Vercel, go to your project settings
2. Click Domains
3. Add: clawscreenwriter.com
4. Vercel gives you DNS records to add
5. Go to your domain registrar (GoDaddy, Namecheap, etc.)
6. Add the DNS records Vercel provides
7. Wait 10–30 minutes for DNS to propagate
8. clawscreenwriter.com is live

STEP 25 Test the Live Site

Open clawscreenwriter.com in your browser.
Confirm everything works identically to local.
Open the site in two browser tabs.
Type in one tab.
Confirm the text appears in the other tab within 1 second.
That is the real-time sync confirming it is all working.

PART 6 Start Writing with OpenClaw

Every Session · Every Project

The platform is live. Now this is your daily workflow for using OpenClaw to write autonomously into your screenplay.

STEP 26 Launch a Writing Session

Every time you want OpenClaw to write, open Terminal and run:

```
cd ~/Desktop/clawscreenwriter
claude
```

Session Starter — Paste This at the Beginning of Every Session

This gives OpenClaw full context about what it is and where everything lives. Paste it once at the start, then give it your writing instructions.

SESSION STARTER — Paste this at the start of every OpenClaw session:

```
You are OpenClaw, an AI screenwriting agent.
You have access to the Claw Screenwriter platform
via Supabase REST API.
All credentials are in openclaw-api.md in this project.

Before writing anything:
1. Read openclaw-api.md for the API details
2. Read the current project's logline and synopsis
3. GET the current script to retrieve its id and content
4. Write or continue the screenplay in Fountain format
5. PATCH the document with the full updated content
6. The writer sees your changes in real time

Fountain format rules:
- Scene headings: INT. or EXT. LOCATION -- TIME
- Character names: ALL CAPS on their own line
- Parentheticals: (in parentheses) after character name
- Transitions: FADE OUT. or SMASH CUT TO:
- Never overwrite existing content unless told to
- Always append new scenes after existing content
```

```
Current project: [YOUR PROJECT NAME]
```

Writing Commands — Use These Once OpenClaw Is Running

What You Want	What to Type
Write an entire act autonomously	"Write Act 1 based on my outline."
Continue from where you left off	"Read the script and write the next three scenes."
Write a specific scene	"Write the scene where Mara discovers the signal is changing. Make it tense."
Write from your outline	"Read my outline and write all scenes in Act 2."
Create a new draft	"Create Draft 2 and rewrite the opening sequence."
Write a development document	"Read my logline and write a full treatment."
Just write and walk away	"Write the complete screenplay based on my synopsis and outline. Go."

TIP

The last command is the most powerful thing this platform does. No other screenwriting tool lets you say Go and come back to a complete draft. That is OpenClaw.

Claw Screenwriter

The platform is built. The agent is ready. Go write.

clawscreenwriter.com · One73 Entertainment LLC